

Project 2: Passive Radar

实验报告

一. 小组成员及分工: (各 25%)

苗超然: Task1

王越: Task1, Task2 作图

陈正非: Task2, 算法优化

罗海宸: PPT, 实验报告

二. 背景介绍:

(一) 多路径传播

信号通过两条或多条路径到达接收天线。因为存在建筑物和山体, 信号可能会被反射, 从而形成多径传播。多径传播导致信号衰减, 限制了宽带和传输速度。接收机接收到的信号是直接波和反射波的叠加波。

(二) 雷达

该装置可以分析物体表面反射的高频无线电波, 以确定物体的位置和速度。

主动雷达: 发射机和接收机都是雷达, 并且离得很近。

无源雷达: 基站作为发射机, 雷达作为接收机。通过直接路径从发射机接收参考信号。然后接收目标的散射信号。计算两个信号样本之间的时间差和频率差。

(三) 多普勒频移

当信号源和观测器相对运动时, 信号的频率会发生变化。接近时增加(称为“蓝移”), 分开时减少(称为“红移”)。波源振动后会发出一个波, 而接收机由于相对运动而接收到不同的频率。

$$f' = \left(\frac{v \pm v_0}{v \mp v_s} \right) f$$

公式 1 频移公式

三. 项目目标

利用被动雷达对目标监测得到的二十组数据, 分析并得到被测目标的距

离与速度。

四. 项目内容

(一) Task1

1. 任务描述:

对 20 组信号分别进行频移, 低通滤波处理, 并分别画出监测信号与参考信号的原始时域频域图、频移后的时域频域图、低通滤波后时域频域图。

2. 实验过程:

(1) 首先将原始数据导入 Matlab, 并画出参考信号, 监测信号 的时域, 频域图。

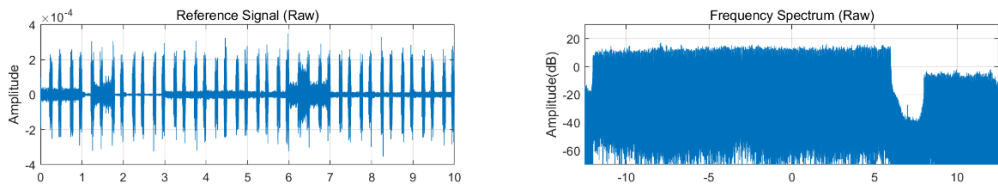


图 1 参考信号

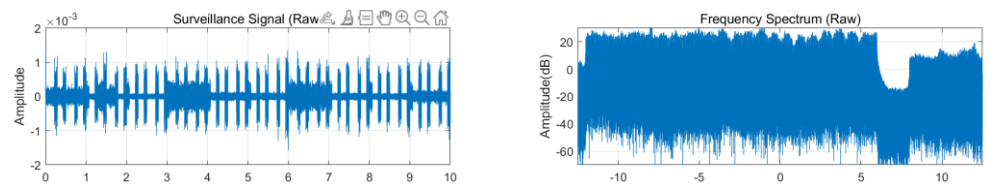


图 2 监测信号

(2) 对两个信号分别做频移处理, 并画出时域频域图。

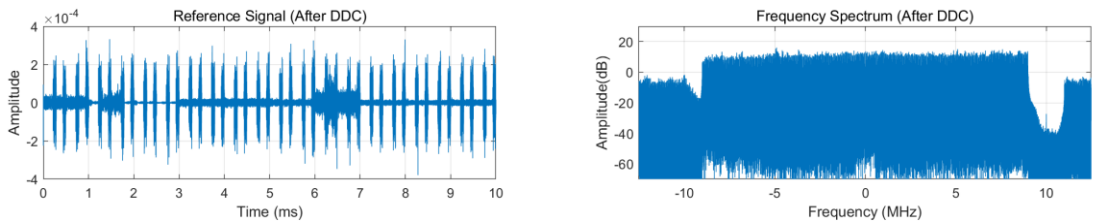


图 3 参考信号 DDC

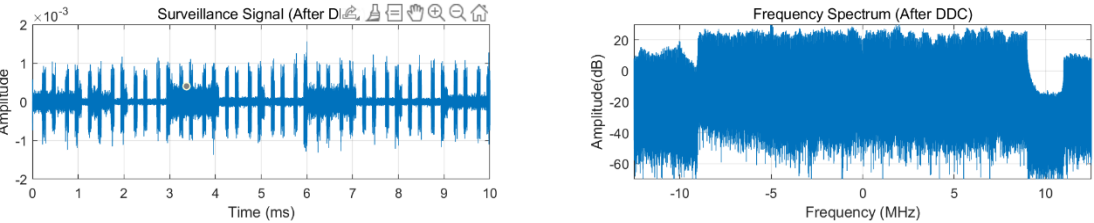


图 4 监测信号 DDC

(3) 对两个信号继续做低通滤波处理, 并画出时域频域图。

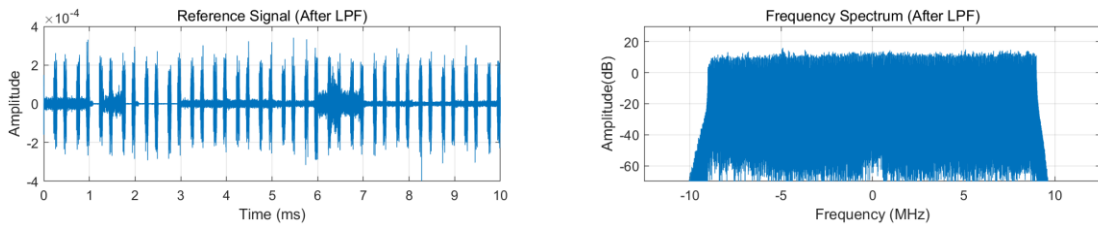


图 5 参考信号经过滤波

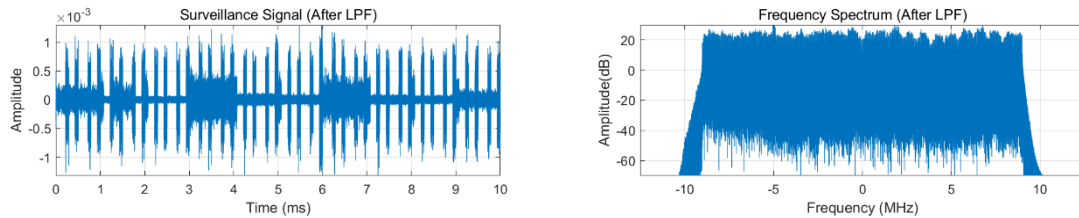


图 6 监测信号经过滤波

3. 关键代码

我们引入了两个函数，`referprocessor(idx_start_time)`，`surveilprocessor(idx_start_time)`，分别用于参考信号与监测信号的数据处理。实现代码如下（以监测信号为例）

图 7 监测信号处理代码

```
function out=referprocessor1(idx_start_time)
addpath('data')
load(sprintf('data\data_%d.mat',idx_start_time));
bandwidth=9e6;
%时域画图
l=length(seq_ref);
ts=1/f_s;
duration_plot=0.01;
num_t_axis_plot=duration_plot*f_s;
t_axis_plot=0:1/f_s:duration_plot-1/f_s;

%频域画图
f=-f_s/2/1e6:4/2/1e6:(f_s-1)/2/1e6;
seq_ref_f = 20 * log10(abs(fftshift(fft(seq_ref))));
%DDC
seq_ref_ddc_t=seq_ref.*exp(-1i*2*pi*f_ddc*(0:duration*f_s-1)/f_s);
seq_ref_ddc_f = 20 * log10(abs(fftshift(fft(seq_ref_ddc_t))));
%LPF
[b,a]=butter(20,bandwidth/(0.5*f_s));
seq_ref_lpf_t= filter(b,a,seq_ref_ddc_t);
seq_ref_lpf_f = 20 * log10(abs(fftshift(fft(seq_ref_lpf_t))));
out=seq_ref_lpf_t;
%画图
figure
plot(t_axis_plot,out)
```

```
function parallelDraw(n)
figure
rs=referprocessor(n);
figure
ss=surveilprocessor(n);
load(sprintf('datas\data_%d.mat',n));
seq_ref=rs;
seq_sur=ss;
save(sprintf('datas\data_%ds.mat',n));
clear;
end
```

(二) Task2:

1. 任务描述:

本任务要求计算出每个 0.5s 内最合适的时间差 τ 和多普勒频移 f_D ，时间差可以反映被检测目标与雷达之间的距离，多普勒频移可以反映被监测目标的运动速度，进而对目标在该时间段内的位置和运动速度做估计，将所有的 0.5s 的时间差和多普勒频移分别取出来，即可得到目标物体在整个时间段内的运动状况。

2. 实验过程:

(1) 利用模糊函数，对每个 0.5s 内的 τ , f_D ，进行遍历，当模糊函数取得最大值时，即是最合适的 τ , f_D 。模糊函数如下：(由于 Matlab 只能进行离散计算，因此在下图中其被转化成离散形式)

(2) 在 Matlab 中 load 进 Task1 中处理好的数据。对 τ , f_D ，进行遍历。由于

$$Cor(\tau, f_D) = \int_t^{t+T} y_{surv}(t) y_{ref}^*(t - \tau) e^{-j2\pi f_D t} dt \quad \longrightarrow \quad Cor(\tau, f_D) = \sum_{n=0}^{N-1} y_{surv}[nT_s] y_{ref}^*[nT_s - \tau] e^{-j2\pi f_D nT_s}$$

公式 2 模糊函数的离散化

采样频率为 f_s , 则 τ 的最小值为 $1/f_s$ ，最小距离分度值为 c/f_s ，计算得 12m。考虑到目标物体运动不超过 100m，因此在这里可以取 0-72m 进行遍历，因此 τ 的取值为 1-6；对于 f_D ，由于 $f_d = \frac{v \cdot f_c}{c}$ ，由于物体运动速度不超过 6m/s，因此 f_d 取值为 -40-40。

(3) 在写 ambiguity 函数时，使用三层 for 循环分别遍历 τ , f_D ，在对 cor 进行求和，函数返回最大 τ , f_D 值。

(4) 最后使用 imagesc () 函数画出对应的 Range-Doppler Spectrum，横坐标为 f_D ，纵坐标为距离，亮度表示模糊函数 $Cor(\tau, f_D)$ 的大小，由此可知最亮的位置即为最佳取值位置。

3. 关键代码：

(1) 模糊函数代码：

```
function [tau_M, fd_M, corM, cors] = ambiguity(data)
load(data);
y_sur = seq_sur;
y_ref_conjugate = conj(seq_ref);
cors = zeros(7,41);
tau = 0:1:6;
fd = -40:2:40;
for i = 1:1:7
    for j = 1:1:41
        for k = 1:1:length(seq_ref)
            if(tau(i)<1)
                y_ref_conj_current=0;
            else
                y_ref_conj_current=y_ref_conjugate((tau(i)));
            end
            cors(i,j)=cors(i,j)+y_sur(k)*y_ref_conj_current*exp(-2i*pi*fd(j)*(k-1)*(1/f_s));
        end
    end
end
corM = max(max(cors));
[tau_M_index, fd_M_index] = find(corM == cors);
tau_M = tau_M_index-1;
fd_M = fd_M_index*2-42;
```

图 8 模糊函数代码（优化前）

在模糊函数结束后将得到的数据保存，指令为：

```
save("data_"+1+"task2.mat");
```

图 9 保存指令代码

(2) 作图代码:

```
for i = 1:1:20
load("data_"+1+"task2.mat");
cors = abs(cors);
x = -40:2:40;
y = 0:1:6;
imagesc(x,12*y,(cors/abs(corsM))),xlabel('Doppler frequency(Hz)');
ylabel('Range(m)');
set(gca,'ytick',0:12:72);
colorbar
axis xy;
title('Range-Doppler Spectrum (0.0-0.5s) 12m 4Hz');
end
```

图 10 作图代码

4. 作图:

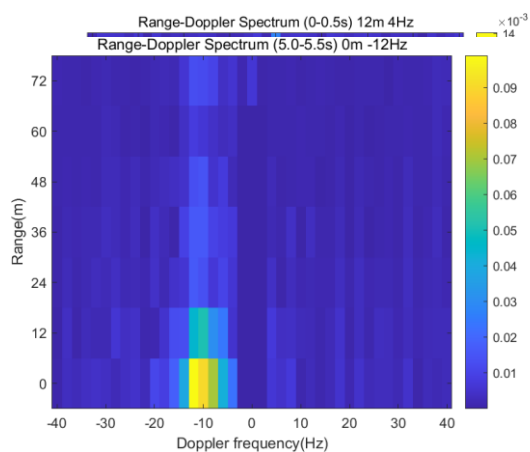


图 11 0~0.5s

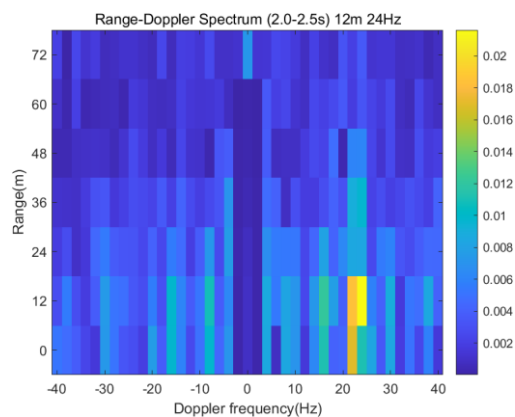


图 12 2~2.5s

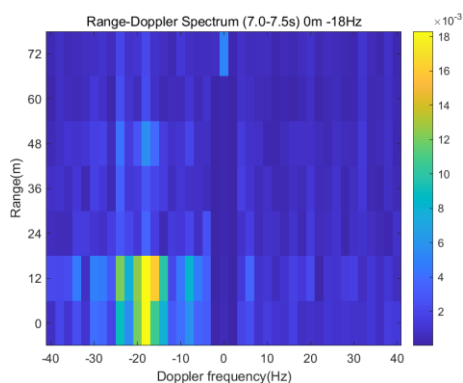


图 13 5~5.5s

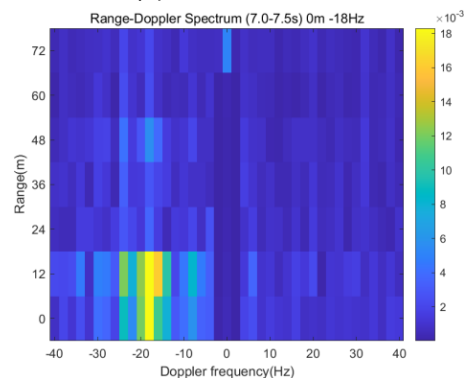


图 14 7~7.5s

5. 进一步处理:

取出每一组数据中 cor 最大的所在行, 就可以得到总的 Time-Doppler Spectrum. 作图代码如下:

```

results = abs(results);
x = -40:2:40;
y = 0:0.1:10;
imagesc(x,y,results),xlabel('Doppler Frequency (Hz)'),ylabel('Time (s)');
set(gca,'ytick',0:0.5:9.5);
set(gca,'xtick',-40:2:40);
colorbar
axis xy;
title("Time Doppler Spectrum");

```

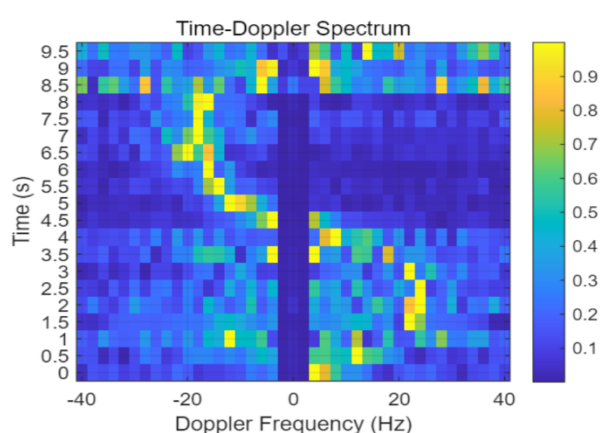


图 15 最终的多普勒-时间图

可以通过时间-多普勒频率图大致得出运动过程，被监测物体经历了先远离后靠近的过程。

(三) 拓展：优化模糊函数算法

1. 优化方式：

由于模糊函数中含有指数项且与傅里叶变换形式相似，因此我们可以尝试使用傅里叶变换 `fft` 来代替原始直接算法。理由：

傅里叶变换公式如下：

$$F(\omega) = \int_{-\infty}^{\infty} f(x)e^{-i\omega x} dx$$

公式 3 傅里叶变换公式

与模糊函数公式相对比，可以看出：

$$f(x) = y_{surv}(t) * y_{ref}^* * u(t)$$

2. 代码实现:

```
offset = round(tau*f_s);
if(mod(offset,2)~=0)
    addition = 1;
else
    addition = 0;
end

t = (1:length(y_surv)+offset+addition)/f_s;

yt_surv = [y_surv,zeros(1,offset)];
yt_ref = [zeros(1,offset),y_ref_conjugate];

y=yt_surv.*yt_ref;

if(mod(offset,2)~=0)
    y = [y,0];
end

n = length(t);
fft_result = fftshift(fft(y));
indices = round(n.*f_d/fs+1+n/2);
z = fft_result(1,indices);
```

图 16 FFT 算法代码实现

3. 运行结果:

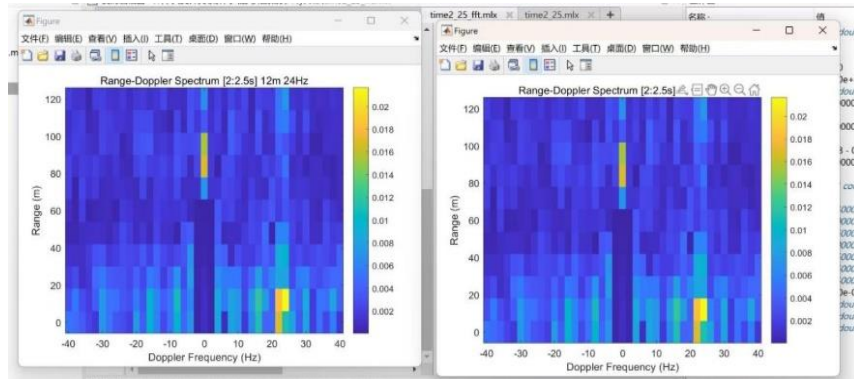


图 17 DWT 循环算法与 FFT 算法结果对比

由对比可知上述两种算法最终效果完全相同。

4. 运算时间对比:

```
tic;
y_ref = y_ref(1,1:0.01*f_s);
y_sur = y_sur(1,1:0.01*f_s);
y_ref_conjugate = conj(y_ref);
cors = zeros(7,41);
tau = 0:1:6;
fd = -40:2:40;
for i = 1:1:7
    for j = 1:1:41
        for k = 1:1:length(y_ref)
            if(tau(i)<1)
                y_ref_conj_current=0;
            else
                y_ref_conj_current=y_ref_conjugate((tau(i)));
            end
            cors(i,j)=cors(i,j)+y_sur(k)*y_ref_conj_current*exp(-2i*pi*fd(j)*(k-1)*(1/f_s));
        end
    end
end
toc;
```

历时 5.757666 秒。

```
tic;
offset = round(0.01*f_s);
if(mod(offset,2)~=0)
    addition = 1;
else
    addition = 0;
end

t = (1:length(y_sur)+offset+addition)/f_s;

yt_sur = [y_sur,zeros(1,offset)];
yt_ref = [zeros(1,offset),y_ref_conjugate];

y=yt_sur.*yt_ref;

if(mod(offset,2)~=0)
    y = [y,0];
end

n = length(t);
fft_result = fftshift(fft(y));
indices = round(n.*fd/fs+1+n/2);
z = fft_result(1,indices);
toc;
```

历时 0.943257 秒。

图 18 时间对比

5. 原理分析:

(1) 由前面实验课知识可知, DWT 的时间复杂度为 $O(n^2)$, 而 FFT 为 $O(n \log n)$, 因此从该角度 FFT 优于 DWT;

(2) DWT 含有大量有关自然对数的指数运算, 对于计算机来说这样的运算量很大, 因此花费时间较长。

6. 结论:

使用 FFT 算法能在保证结果正确的前提下大幅度降低运算时间, 不失为一种高效的方法。

五. 实验收获

1. 了解了被动雷达的监测原理, 并学会通过 matlab 对监测得到的数据进行处理, 最终分析物体的位置, 运动信息。

2. 通过小组分工合作的模式完成本次项目, 增强了小组的团队凝聚力与个人的协调沟通能力, 为未来的团队任务打下坚实的基础。